DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Advanced Data Structures & Algorithms — Course Project

Digital Logic Circuit Simulator

A Graph-Theoretic Framework for Combinational Logic Validation & Simulation

Presented by:

Markanda Meher (Roll No. 206126020) | Nikhil (Roll No. 206126021)

First Review — October 12, 2026

Physical Digital Circuit Prototyping Bottlenecks

Hardware Prototyping Constraints

- **Discrete Breadboard Congestion:** Constructing multi-gate combinational designs requires dozens of physical jumper wires. Loose pin contacts, contact bounce, and wiring clutter frequently obscure connectivity.
- **Absence of Automated Verification:** Physical breadboards lack any design-rule checking mechanism. Illegal configurations—such as unwanted circular feedback loops or shorted gate outputs—are undetectable prior to powering the circuit.
- **Component Damage & Resource Waste:** Accidental reverse polarity, inverted IC pin insertion, or shorted VCC/GND traces permanently destroy 7400-series TTL/CMOS integrated circuits, incurring recurring laboratory replacement costs.
- **Tedious Manual Fault Localization:** Isolating a single faulty connection or burned gate requires slow, manual node-by-node probing with multimeters or oscilloscopes.

Academic & Scalability Impact

- **High Cognitive Debugging Overhead:** Students and engineers often spend over 70% of experimental laboratory sessions troubleshooting physical wire loose ends rather than studying digital logic and computer architecture fundamentals.
- **Strict Scalability Ceiling:** Scaling beyond 15–20 logic gates (e.g., 4-bit Arithmetic Logic Units, multiplexer trees, or multi-bit adders) becomes physically impractical on educational breadboard layouts.
- **Need for Pre-Silicon Modeling:** Modern VLSI engineering mandates software verification before hardware fabrication. An educational simulator bridges this gap by demonstrating how algorithmic models replace costly physical trial-and-error.

The Graph-Theoretic Software Simulation Paradigm

Mathematical Graph Abstraction

- **Directed Graph Formalism:** A digital combinational circuit is mathematically modeled as a Directed Acyclic Graph G = (V, E), where logic gates are vertices and interconnecting wires are directed edges.
- **Strict Signal Causality:** Directed edges enforce physical signal propagation—signals flow strictly from driver output pins into load input pins with zero ambiguity.
- **Decoupling Geometry from Logic:** Abstracting physical wire resistance, capacitance, and mechanical wear allows engineers to focus purely on functional correctness and topological structure.

Simulation Capabilities & Objectives

- **Automated Cycle Trapping:** Traverses the graph in linear time O(V + E) to detect feedback loops before simulation begins, preventing infinite oscillation in combinational circuits.
- **Topological Execution Ordering:** Computes a valid evaluation schedule, guaranteeing every gate computes only after all its input dependencies have settled, eliminating race conditions.
- **Exhaustive Truth Table Generation:** Simulates all 2^N input combinations automatically, verifying logical equivalence against theoretical Boolean algebra in milliseconds.
- **Safe, Zero-Cost Experimentation:** Enables students to freely modify netlists, probe individual gate pins, and validate complex architectures with zero hardware costs.

In-Degree Topological Sorting (Kahn, 1962)

Citation: A. B. Kahn, "Topological Sorting of Large Networks", Communications of the ACM, Vol. 5, No. 11, pp. 558–562, Nov. 1962.

Analysis of Algorithmic Contents & Contributions:

- Introduced the in-degree reduction queue paradigm for establishing linear execution schedules in Directed Acyclic Graphs (DAGs).
- Operates in optimal linear time O(V + E) by tracking in-degrees and enqueuing vertices whose dependencies reach zero.
- Forms the foundational algorithmic basis for dependency resolution in compilers, task schedulers, and EDA netlist simulation.

Critical Drawbacks & Limitations for Circuit Simulation:

- **Silent Failure on Cycles (No Localization):** If the netlist contains a feedback loop, Kahn's algorithm halts prematurely when the queue becomes empty with unprocessed gates remaining. Crucially, it cannot pinpoint which specific back-edge or gates formed the cycle.
- **Arbitrary Tie-Breaking:** When multiple gates reach in-degree zero simultaneously, Kahn's algorithm dequeues in arbitrary queue order, failing to organize gates into levelized parallel stages.
- **Static In-Degree Assumption:** Assumes graph topology is completely static; modifying or deleting a wire requires full re-computation of all vertex in-degrees from scratch.

DFS & Linear Graph Connectivity (Tarjan, 1972)

Citation: R. E. Tarjan, "Depth-First Search and Linear Graph Algorithms", SIAM Journal on Computing, Vol. 1, No. 2, pp. 146–160, June 1972.

Analysis of Algorithmic Contents & Contributions:

- Formalized Depth-First Search using 3-state node classification (White = unvisited, Gray = active in call stack, Black = fully explored).
- Established linear-time O(V + E) algorithms for detecting strongly connected components (SCC), articulation points, and cycle back-edges.
- Introduced discovery/finishing timestamps and low-link tracking for deep structural graph decomposition.

Critical Drawbacks & Limitations for Circuit Simulation:

- **Recursion Call-Stack Overhead:** Standard recursive implementations allocate O(V) stack frames. In deep combinational ripple circuits or large-scale netlists, this risks system stack overflow.
- **No Forward Evaluation Schedule:** While DFS back-edges isolate cycles perfectly, the resulting reverse post-order traversal is inherently backward-directed and does not yield levelized forward simulation stages.
- **Single-Pin Node Simplification:** Classical graph DFS treats each vertex as an atomic scalar entity, failing to handle multiple input pins with distinct logical positions (e.g., MUX select vs data pins).

Disjoint-Set Union & Net Equivalence (Tarjan, 1975)

Citation: R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm", Journal of the ACM, Vol. 22, No. 2, pp. 215–225, April 1975.

Analysis of Algorithmic Contents & Contributions:

- Analyzed the Disjoint-Set Union (DSU) data structure combining union-by-rank and path compression heuristics.
- Proved the theoretical amortized operational complexity of $O(m \cdot \alpha(m, n))$, where α is the inverse Ackermann function (effectively ≤ 4 for all practical values).
- Widely utilized in EDA netlist parsing to group connected electrical pins into unified equipotential wire nets.

Critical Drawbacks & Limitations for Circuit Simulation:

- **Symmetric / Undirected Equivalence Only:** DSU partitions vertices into undirected equivalence classes. It cannot enforce directional signal propagation (it cannot distinguish a driving gate output pin from driven load gate inputs).
- **Irreversible Union Operations:** Standard DSU does not support set splitting. If a user deletes a wire connection between two gates, the entire disjoint set forest must be rebuilt from scratch in $O(E)$ time.
- **No Execution Ordering:** DSU tracks connectivity only; it provides zero capability for scheduling Boolean gate evaluations.

Comparative Evaluation: Surveyed Algorithms vs. Proposed Simulator

Evaluation Metric	Kahn (1962)	Tarjan DFS (1972)	Tarjan DSU (1975)	Our Simulator
Primary Algorithmic Purpose	Topological Ordering via In-Degree Queue	Graph Traversal & SCC Decomposition	Disjoint-Set Net Connectivity	**End-to-End Logic Simulation & Verification**
Time Complexity	O(V + E) linear	O(V + E) linear	O(m · α(n)) near-linear	**O(V + E) linear**
Cycle Localization	No (halts silently when queue empties)	Yes (detects back-edges)	N/A (undirected sets)	**Yes (isolates & logs exact cycle path)**
Execution Scheduling	Yes (valid linear topological order)	Requires reverse post-order traversal	No (tracks connectivity only)	**Yes (guaranteed race-free evaluation)**
Directional Flow & Logic Eval	Directional only (no logic evaluation)	Directional only (no logic evaluation)	Undirected only (no signal direction)	**Directional + Multi-gate Boolean truth tables**

4-Stage Simulation Architecture & Dataflow Pipeline

Stage 1: Netlist & Graph Model
Builds Directed Graph G=(V,E)
Adjacency List & In-Degrees
O(1) Hash Map Gate Indexing

→

Stage 2: Cycle Detection Engine
Tarjan's 3-Color DFS
White/Gray/Black State Machine
Traps & Isolates Back-Edges

→

Stage 3: Topological Scheduler
Kahn's In-Degree Queue BFS
Resolves Data Dependencies
Produces Linear Order Array

→

Stage 4: Execution & Verification
Boolean Gate Switch Engine
Level-by-Level Signal Propagation
Exhaustive 2^N Truth Table

Pipeline Invariants & Execution Flow:

- **Invariant 1 (Strict Pre-Simulation Validation):** Execution is strictly blocked if Stage 2 detects a cycle. Feedback loops are halted with the exact back-edge reported, preventing infinite simulation loops.
- **Invariant 2 (Dependency Resolution Invariance):** Stage 3 guarantees that every gate in the execution array appears strictly after all its predecessor driver gates, eliminating race conditions in O(V + E) time.
- **Invariant 3 (Exhaustive Verification):** Stage 4 iterates across all 2^N input permutations, executing the circuit topologically and validating correctness against standard truth tables.

Algorithmic Novelties & Engineering Contributions

1. Decoupled Dual-Phase Graph Validation

Standard simulators typically rely either on Kahn's algorithm alone (which fails blindly on cycles without reporting the fault location) or DFS alone (which does not provide a levelized forward execution queue). Our contribution decouples cycle diagnosis from evaluation scheduling: Tarjan's 3-Color DFS is used purely for cycle localization, while Kahn's algorithm is applied strictly to validated DAGs.

2. Dependency-Guaranteed Race-Free Execution in O(V + E)

In physical combinational circuits, signal race conditions occur when downstream gates calculate before upstream inputs settle. Our topological scheduler enforces strict dependency invariance, guaranteeing that every gate evaluates only after all its driver gates are resolved in linear O(V + E) time.

3. High-Performance O(1) Hash Map Gate Indexing

Rather than relying on continuous O(V) array scans, our simulator indexes gates, input pins, and wire nets via hash maps with unique string identifiers. This enables instant gate state updates and predecessor lookups during signal propagation.

4. Automated Exhaustive Bitwise Verification Engine

Implements an automated truth table generator evaluating all 2^N input vectors. Computes expected Boolean formulas against simulated outputs, achieving complete functional verification in milliseconds.

Module 1 (Graph Netlist) & Module 2 (3-Color DFS Cycle Detection)

Algorithm 1: Netlist Graph Construction

```
Input:  Gates V = {g_1, g_2, ...}, Wires E = {(u, v), ...}
Output: Adjacency list adjList, In-degree table inDegree

1:  adjList  = HashMap<GateID, List<GateID>>()
2:  inDegree = HashMap<GateID, Integer>()
3:  For each gate g in V do:
4:      adjList[g.id] = empty_list()
5:      inDegree[g.id] = 0
6:  End For
7:  For each wire (u, v) in E do:
8:      // u is driver gate, v is driven input
9:      adjList[u].append(v)
10:     inDegree[v] = inDegree[v] + 1
11: End For
12: Return (adjList, inDegree)

Complexity: O(V + E) time, O(V + E) space
```

Algorithm 2: Tarjan's 3-Color DFS Cycle Detection

```
Input:  Current gate u, Color map color, adjList
Output: Boolean (TRUE if cycle exists, FALSE otherwise)

1:  color[u] = GRAY  // Gate in active recursion stack
2:  For each successor v in adjList[u] do:
3:      If color[v] == GRAY then:
4:          RecordCycleEdge(u, v)
5:          Return TRUE  // Back-edge found: Cycle trapped!
6:      End If
7:      If color[v] == WHITE then:
8:          If DetectCycle(v, color, adjList) == TRUE then:
9:              Return TRUE
10:         End If
11:     End If
12: End For
13: color[u] = BLACK // Subtree verified acyclic
14: Return FALSE

Complexity: O(V + E) time, O(V) recursion stack
```

Module 3 (Kahn's Scheduler) & Module 4 (Logic Execution)

Algorithm 3: Kahn's In-Degree Topo Scheduler

```
Input:  Gates V, In-degree table inDegree, adjList
Output: Topologically ordered execution list evalOrder

1:  Q = EmptyQueue()
2:  For each gate g in V do:
3:      If inDegree[g] == 0 then: Q.push(g)
4:  End For
5:  evalOrder = []
6:  While Q is not empty do:
7:      u = Q.pop()
8:      evalOrder.append(u)
9:      For each successor v in adjList[u] do:
10:         inDegree[v] = inDegree[v] - 1
11:         If inDegree[v] == 0 then: Q.push(v)
12:     End For
13: End While
14: Return evalOrder

Complexity: O(V + E) time, O(V) queue memory
```

Algorithm 4: Gate Simulation & Truth Table Engine

```
Algorithm SimulatePass(evalOrder, inputVector):
1:  values = HashMap<GateID, Boolean>()
2:  For each gate g in evalOrder do:
3:      If g.type == INPUT then:
4:          values[g.id] = inputVector[g.id]
5:      Else:
6:          inVals = [values[p] for p in g.predecessors]
7:          values[g.id] = EvaluateGateLogic(g.type, inVals)
8:      End If
9:  End For
10: Return values

Algorithm GenerateTruthTable(N_inputs, evalOrder):
1:  For row = 0 to (2^N - 1) do:
2:      inVec = DecodeBinaryRow(row, N_inputs)
3:      outVec = SimulatePass(evalOrder, inVec)
4:      PrintTruthTableRow(inVec, outVec)
5:  End For
```

5. DATASET & BENCHMARK SUITE

IEEE ISCAS '85 Benchmark Circuit Suite

Benchmark Origin & Download Links

- **Benchmark Suite:** IEEE ISCAS '85 (International Symposium on Circuits and Systems) Benchmark Suite.
- **Industry Standard Role:** The universally recognized reference standard in Electronic Design Automation (EDA) and academic CAD research for combinational circuit validation and test pattern generation.
- **Primary Repository Link:** University of Tennessee EECS CAD Lab: https://web.eecs.utk.edu/~troy/benchmark/
- **Mirror Repository Link:** Czech Technical University (FEE CTU): https://ddd.fit.cvut.cz/prj/Benchmarks/
- **Netlist File Syntax (.bench):** Human-readable ASCII grammar defining Primary Inputs (`INPUT(x)`), Primary Outputs (`OUTPUT(y)`), and logical equations (e.g., `10 = NAND(1, 3)`).

Benchmark Netlists & Project Role

- **c17 (6 Gates, 5 Inputs, 2 Outputs):** Smallest ISCAS netlist; used for fundamental sanity checks and exhaustive 32-row truth table verification against IEEE golden output specifications.
- **c432 (160 Gates, 36 Inputs, 7 Outputs):** 27-channel interrupt controller; evaluates topological scheduling across medium-scale multi-fanout logic trees.
- **c499 (202 Gates, 41 Inputs, 32 Outputs):** 32-bit single-error-correcting circuit (SEC-DED); verifies deep logic paths and XOR/NAND propagation.
- **c880 (383 Gates, 60 Inputs, 26 Outputs):** 8-bit Arithmetic Logic Unit (ALU); validates linear O(V + E) runtime scaling and memory efficiency.
- **Validation Utility in Review:** Demonstrates algorithmic correctness against industry standard benchmarks rather than ad-hoc custom circuits.